

Project Report: Housing Price Prediction

Dataset: Housing Prices Competition for Kaggle Learn Users

1. Project Objective

The goal of this project was to develop a Machine Learning model capable of estimating residential sale prices using the 'Housing Prices Competition for Kaggle Learn Users' dataset. The project focused on maximizing predictive accuracy through advanced preprocessing and data engineering techniques.

2. Exploratory Data Analysis (EDA) and Data Cleaning

- **Correlation Analysis:** I calculated the correlation coefficients between numerical variables and the target variable (*SalePrice*).
- **Data Visualization:** I generated scatter plots for high-correlation features (> 0.6), such as *GrLivArea* (Above Grade Living Area) and *OverallQual* (Overall Quality).
- **Outlier Treatment:** Through visualization, I identified statistical anomalies (outliers)—specifically, houses with exceptionally large square footage sold at unusually low prices. Removing these was necessary to prevent the model from becoming biased or distorted.

3. Data Preprocessing and Pipeline Architecture

To ensure an efficient workflow and prevent **data leakage**, I constructed a robust **Pipeline** that included:

- **Data Imputation:** Handling missing values for both numerical and categorical variables.
- **Encoding:**
 - **One-Hot Encoding:** Applied to nominal variables (those without an intrinsic order).
 - **Ordinal Encoding:** Applied to variables representing a quality hierarchy (e.g., Poor, Average, Good, Excellent).
- **Data Splitting:** Dividing the dataset into Training and Validation subsets to evaluate performance on unseen data.

4. Feature Engineering

This stage provided the most significant improvement to the model's score:

- **Creating New Variables:** I synthesized new features such as '**TotalSF**' (Total Square Footage) by combining basement and upper-floor areas.
- **Feature Selection:** I employed feature selection techniques to retain only variables with an importance score higher than 0.001, effectively removing the "noise" caused by low-impact columns.

5. Advanced Model Optimization (Meta-heuristics)

To overcome the limitations of classical optimization methods, I implemented an advanced tuning stage:

- **Genetic Algorithms (PyGAD):** I utilized genetic algorithms to optimize the hyperparameters of the XGBoost model. This meta-heuristic approach allowed for the exploration of a complex search space, identifying a parameter combination (learning rate, max_depth, n_estimators) that outperformed standard techniques like GridSearch.

6. Deployment and Interactive Interface

The project was transformed from a static model into a functional web application:

- **Streamlit Framework:** I developed an interactive interface that allows users to input house characteristics (square footage, quality, number of rooms) and receive an instantaneous price estimation.
- **Model Persistence:** The preprocessing pipeline and the optimized model were serialized using joblib, ensuring consistency between the training phase and the real-time prediction phase.

7. Containerization (DevOps Integration)

To ensure system portability and stability:

- **Dockerization:** The application was packaged into a Docker container using a python:3.11-slim base image.
- **Environment Standardization:** By utilizing a Dockerfile and a requirements.txt file, I eliminated the "it works on my machine" issue, enabling the application to run on any server with a single command.

- **Optimization:** The image was optimized for size and speed by installing only the essential dependencies required for execution.

8. Final Results and Conclusions

- **Performance:** The final model based on Genetically Optimized XGBoost demonstrated high generalization capability (GridSearch MAE $\approx 16,500$ vs. Genetic Algorithm MAE $\approx 14,800$).
- **End-to-End Solution:** The project covers the entire lifecycle of an AI product: from exploratory data analysis and data cleaning to feature engineering, mathematical optimization, and final delivery via containerized infrastructure.
- **Future Work:** Planned improvements include integrating a database for storing predictions and implementing a price monitoring system to detect data drift and determine when the model requires retraining.